Claude Code Prompts

One slice per session. Start each with `/clear`, switch to plan mode (Shift+Tab), paste the prompt, review the plan, approve. Before clearing, ask Claude Code to record decisions in `docs/DESIGN.md`. Commit after each slice passes its gate.

Order: Kickoff → H1–H5 (hackathon) → P1–P8 (platform), as in `docs/SPEC.md` §12.

Before the kickoff:

- `CLAUDE.md` at the repo root; `docs/SPEC.md` and `docs/PROMPTS.md` in `docs/`.
- The Case 1 brief at `docs/case1-brief.pdf` and the judges' workbook template at `data/case1-template.xlsx`.
- Optional: the components map as one wide image per flow in `docs/` (the current PDF crops every diagram after the Context column).
- Your design skill folders in `.claude/skills/` (each with its `SKILL.md`).
- Keys in `.env` (never in chat): `GEMINI_API_KEY`, `GOOGLE_MAPS_API_KEY` (restricted by HTTP referrer for the browser key; a separate server key for Places and Geocoding), database URL. Verification vendor keys come later, in sandbox mode.
- Fill in the deadline and team line in the kickoff prompt below.

Kickoff (no code yet)

```
Deadline: <date and time>. Team: <people and hours each>.

Read CLAUDE.md, docs/SPEC.md (especially §6, §6A, §12), docs/case1-brief.pdf,
data/case1-template.xlsx and every SKILL.md in .claude/skills/. Inspect the repository:
structure, code, tests, config, migrations, .env.example. Don't assume it is empty. If
the brief or template is missing, say so and use only clearly labelled provisional
fixtures; never invent sheet or column names.

Do not write application code, change a schema, call a live service or deploy. Produce
docs/DESIGN.md, at most about 15 pages, with the requirements matrix in
docs/TRACEABILITY.md (source and section, requirement, priority: hackathon / MVP /
future / out of scope, solution, acceptance test, status).

DESIGN.md covers:
1. Summary: what the demo shows, what waits until after the hackathon, and how it fits
the deadline and team.
2. Conflicts: every difference between the Case 1 brief, SPEC and CLAUDE.md, with the
consequence, your recommendation and whether I need to approve it. Don't reconcile
anything silently.
3. Workbook: exact sheets and columns from the template, column mapping, validation,
PINs read as text, whole-workbook rejection vs per-row problems, contradictory fields.
4. Screening: gates, score weights and ranges, missing-data treatment, outcome rules,
with worked examples (incomplete, contradictory, competing, clean). Mark every weight
or threshold the sources don't define as "needs approval".
5. Allocation: territory matching, slots, priority, occupied PINs, distance using PIN
centroids (name the dataset), tie-breaks that don't depend on row order.
6. Follow-ups: how a question maps to a record; how a new fact becomes a proposed field
```

```
change, when to ask for clarification, the user's confirmation, recomputing the whole
territory, showing knock-on changes to other applicants, and the audit log.
7. "Better inputs never worsen status": what it means, its exceptions, and the tests.
8. Onboarding: templates, rules per outcome, experience and language, owners, what
counts as done, approval gates.
9. Gemini: current stable model IDs (check ai.google.dev/gemini-api/docs/models; record
the date), each task's JSON schema, the numberGuard normalisation and allowlist, the
retry-then-fail-safe path, mock mode.
10. Demo deployment: per-session workspaces and their expiry, upload validation,
secrets handling, hosting choice and blockers. Keep it to the fewest services that meet
the acceptance criteria; justify anything extra.
11. Slices H1–H5: deliverables, tests, gate and exclusions for each; which P-slice
items, if any, the hackathon needs.
12. Top 5 risks and a decision register with your recommended defaults (marked as not
yet approved).
13. A judge walkthrough script, step by step.

Then check DESIGN.md against TRACEABILITY.md, list missing inputs and approval gates,
and stop. Approving the design does not approve individual scoring rules, thresholds,
schemas or integrations; those keep their own gates.
```

H1 — Foundation

```
Implement slice H1 from SPEC §12 and DESIGN.md: repo layout, the web app and engine as
decided in DESIGN.md, per-session demo workspaces with expiry, the audit log, and the
app shell.

Build the UI strictly from the design skills in .claude/skills/: set up their tokens
and components first, then the shell (upload screen, applicant list, empty states).
Write cross-session isolation tests: one session must not read or change another's
workbook, records or history.
```

H2 — Ingest and screen

```
Implement slice H2 from SPEC §6 and §6A and DESIGN.md: parse the judges' workbook
exactly as the template defines it, validate it, read PINs as text, load PIN centroids
from the open PIN directory, and run the approved gates and score to produce an outcome
per applicant with evidence, gaps and next steps. Gemini only extracts facts and flags
contradictions from free-text notes, with schema validation and a safe failure path.

Gate: tests on the template workbook, on incomplete and contradictory rows, and the
same result on re-run.
```

H3 — Allocate

```
Implement slice H3 from SPEC §6A and DESIGN.md: territory matching by PIN coverage,
open slots, expansion priority, existing stores as occupied PINs, one store per PIN,
minimum distance → human review, and the approved tie-breaks. Explain every case with
no territory or no slot.
```

```
Gate: allocation tests for competing applicants, exhausted slots, occupied PINs,
missing PIN mappings, and identical results with the rows shuffled.
```

H4 — Follow-ups and onboarding

```
Implement slice H4 from SPEC §6, §6A and §8 and DESIGN.md: answers to status questions
from the stored record and reasons; new facts turned into a proposed field change,
confirmed by the user, then a recompute of the whole territory with before/after for
every affected applicant, logged in history; Gemini explanations through numberGuard;
onboarding plans per outcome, experience, site readiness and language, with each item
marked suggested, outstanding, completed or needs brand approval. Every applicant
message carries the fraud notice.

Gate: follow-up tests ("Why is APP-007 on hold?", "APP-012 has found a ground-floor
space"), the better-inputs tests with their documented exceptions, and zero numberGuard
failures on a full run.
```

H5 — Demo

```
Implement slice H5: polish the end-to-end flow (upload → outcomes and allocation →
follow-ups → onboarding) per the design skills, and deploy it to a public URL with no
login, mock verification only, no secrets in the client.

Gate: run the judge walkthrough from DESIGN.md on the public URL, from a fresh browser
session, and list anything checked by hand.
```

P1 — Tenancy

```
Implement slice P1 from SPEC §12: auth with brand workspaces and roles, Postgres row-
level security, and migration of the demo's data model to tenants. Write cross-tenant
isolation tests that try to read another brand's data at the API and database level.
```

P2 — Data layer

```
Implement slice P2 from SPEC §7 and DESIGN.md: loaders for census population and
households and MoSPI HCES consumption data. Build area profiles: MPCE level, spending
mix by item group, food share (Engel coefficient), and trend, each with source, date,
geographic level and confidence. Anything finer than state × rural/urban is a labelled
estimate.

Add the Maps provider: geocoding and Places-based competitor counts by category and
radius, respecting Google's caching terms (place IDs only; lat/lng at most 30 days;
counts are derived values). Produce a coverage report: which PINs have which data, at
what level and confidence.
```

P3 — Flow A: recommend, then refine

```
Implement SPEC §3, §4 and §7: the brand profile, the scoring engine with every factor including the Engel's Law fit, guardrails (one per PIN, minimum distance, exclusions), ranking, internal payback ranges, and run versioning with side-by-side comparison.

Add Gemini explanations and "what's missing" suggestions with structured output and numberGuard. Build the map screen with the Google Maps JS API: ranked PINs, factor breakdown panel, existing stores and cannibalisation radius, competitor layer, and the refine loop. Follow the design skills.

Gate: scoring unit tests (including Engel ordering for a staples brand vs a premium brand), zero numberGuard failures on a full run, and a recorded walkthrough script.
```

P4 — Flow B: Change Radar

```
Implement SPEC §5. Build source connectors (start with the Gazette of India, GST notifications, one state gazette, MoSPI and CPI releases, and brand-uploaded research), the daily job, dedupe and tagging, Gemini classification with Search grounding and structured output, the Change Impact Score in deterministic code, alerts, the weekly digest, and the "accept proposed re-score" action.

Build a labelled eval set of at least 100 past changes (relevant or not, direction, magnitude) and report precision and recall. Every alert must show its source link and date.
```

P5 — Flow C: applications intake

```
Implement SPEC §6 steps 1–2: brands open locations to applicants, a form builder with category defaults, the applicant portal (mobile-first, in the applicant's language), per-check consent records, and the fraud notice on every applicant message. Applications from the portal must go through the same screening and allocation code as the Case 1 workbook.

Use mock verification adapters only. Test the full path from opening a location to a submitted application.
```

P6 — Verification

```
Implement SPEC §6 step 3: DigiLocker via the chosen licensed vendor (sandbox), bank account verification (sandbox), and the lender-partner attestation interface (mock until the partner is confirmed; partner chosen per brand). Store attestations, never raw statements. Verified funds feed the existing screening gates.

Gate: the Case 1 suite stays green with verification switched on.
```

P7 — Decisions and dashboard

```
Implement SPEC §6 step 6 and §10: the brand's review screen, approval actions (the only path that sets "Approved"), and the brand dashboard.
```

```
Gate: a full walkthrough with a pilot brand's sample data, from profile to an approved
partner with an onboarding plan.
```

P8 — Hardening

```
Implement slice P8: security review (auth, webhooks, keys, row-level security, file
uploads), DPDP retention and deletion jobs, Gemini and Maps cost monitoring per brand,
rate limits, and a load test. List findings with fixes.
```